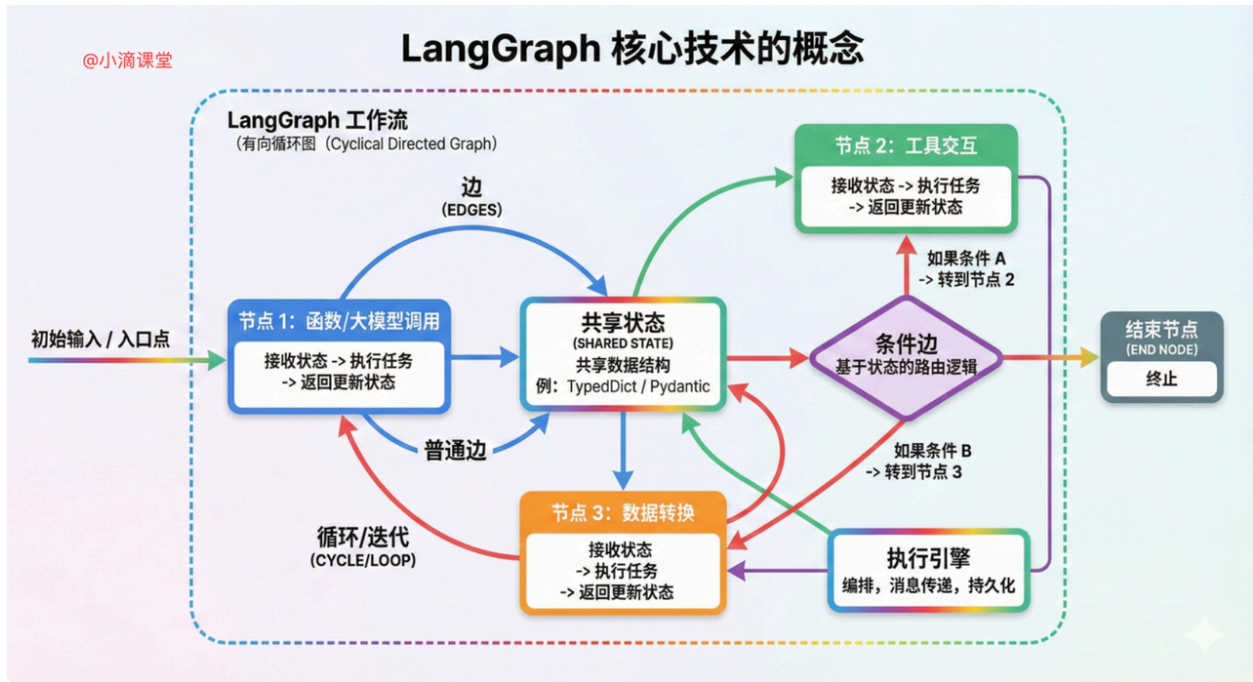


AI开发基础：Langgraph框架从入门到实战开发智能体-附带完整可运行代码



LangGraph 实战教程：从小白到高手

一份让你真正学会用 LangGraph 构建 AI Agent 的实战指南

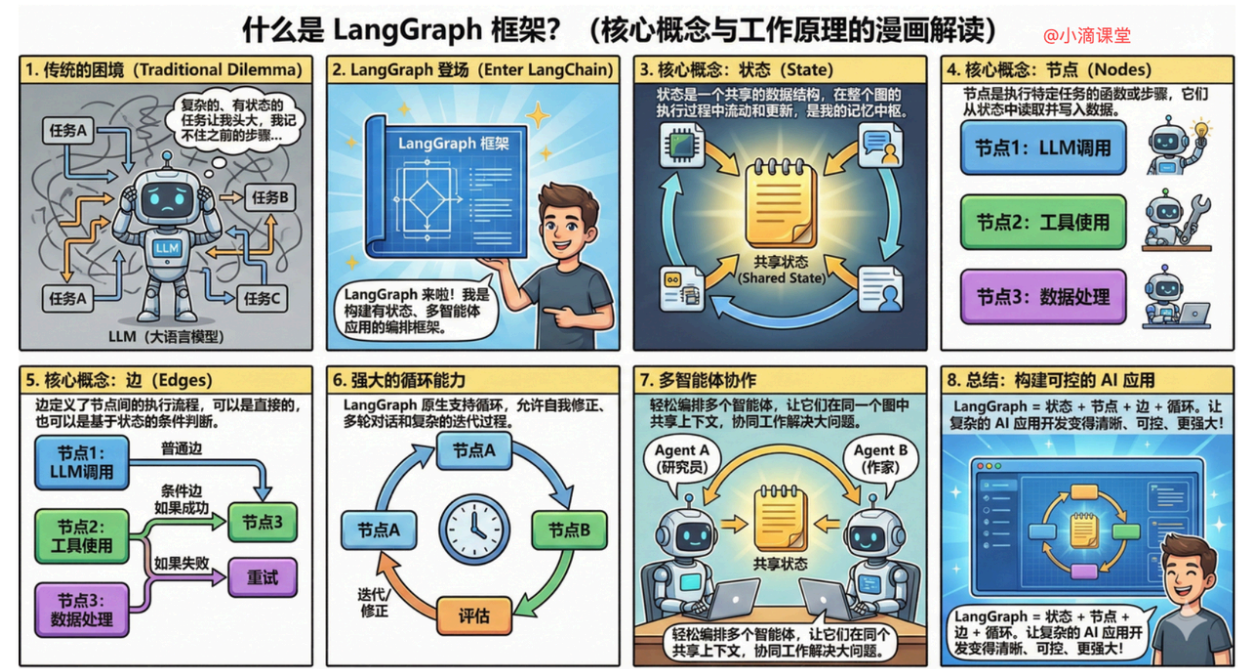
写在前面

🔥 如果你想用大语言模型（LLM）做点有意思的事情，比如做一个能自己查资料、能记住上下文、能调用各种工具的智能助手，那 LangGraph 绝对是你需要了解的东西。

这份教程不会堆砌一堆概念把你绕晕，而是用**真实可运行的代码**带你一步步搞懂这玩意儿到底怎么用。

直接可以运行的代码已经整理好了，需要的话找老师拿

第一章 LangGraph 是什么



一句话解释

LangGraph 就是用来编排 AI 工作流的框架，让你可以把复杂的 AI 任务拆成一个个小步骤，然后像搭积木一样把它们连起来。

为什么需要它？

假设你要做一个客服机器人：

1. 先要判断用户问的是技术问题还是销售问题
2. 技术问题交给技术 Agent 处理
3. 销售问题交给销售 Agent 处理
4. 如果答案质量不够好，还得循环改进

这种有分支、有循环、有状态的复杂流程，用普通代码写起来会很乱。LangGraph 就是专门解决这个问题的。

LangGraph vs LangChain

很多人搞不清这俩的关系：

特性	LangChain	LangGraph
定位	通用 LLM 工具库	工作流编排框架
流程	线性链条 (A→B→C)	图结构（支持分支、循环）
状态	需要手动管理	自动管理
适合场景	简单的问答、RAG	复杂的 Agent、多步骤任务

简单说：LangChain 是做零件的，LangGraph 是用这些零件搭建复杂系统的。

第二章 环境准备

安装依赖

代码块

```
1  # 核心依赖
2  pip install langgraph langchain-core langchain-community
3
4  # 如果要用通义千问
5  pip install dashscope
6
7  # 如果要用 OpenAI
8  pip install langchain-openai
```

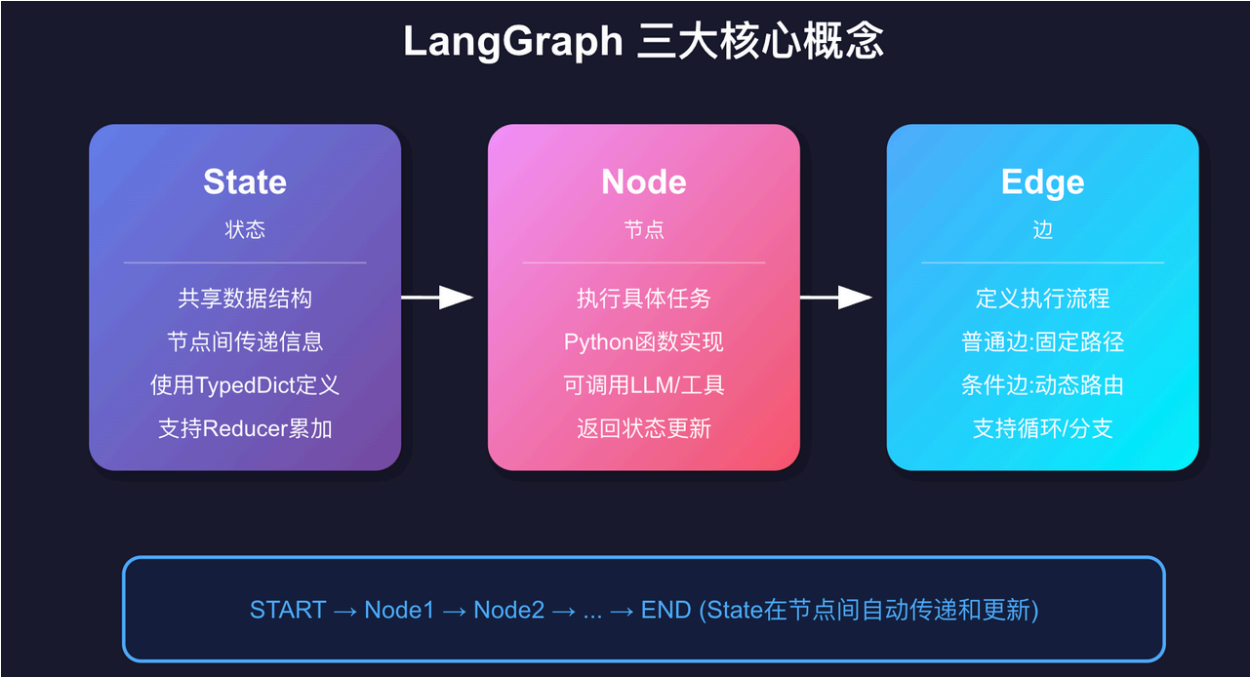
设置 API Key

代码块

```
1  import os
2
3  # 通义千问
4  os.environ["DASHSCOPE_API_KEY"] = "你的key"
5
6  # 或者 OpenAI
7  os.environ["OPENAI_API_KEY"] = "你的key"
```

第三章 核心概念

LangGraph 就三个核心概念，搞懂了这三个，其他都是组合拳。



1. State（状态）

State 就是一个在所有节点间共享的数据结构。你可以把它想象成一个"快递包裹"，每经过一个节点，就往里面加点东西或者修改点东西。

代码块

```
1  from typing_extensions import TypedDict
```

```
2
3 class MyState(TypedDict):
4     count: int          # 计数器
5     history: list[str]  # 历史记录
```

2. Node（节点）

Node 就是**干活的**。每个节点是一个 Python 函数，接收当前状态，返回需要更新的字段。

代码块

```
1 def my_node(state: MyState):
2     # 读取当前状态
3     current_count = state["count"]
4
5     # 返回要更新的字段（不用返回所有字段！）
6     return {
7         "count": current_count + 1,
8         "history": state["history"] + ["执行了一次"]
9     }
```

3. Edge（边）

Edge 定义**节点之间怎么连接**，决定执行顺序。

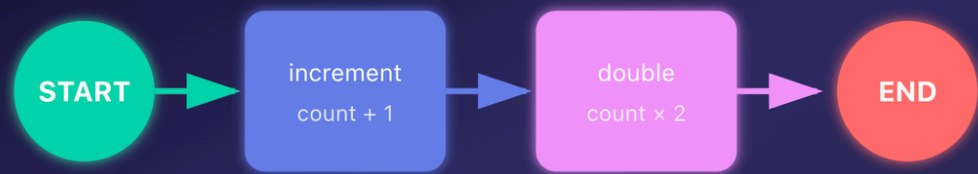
代码块

```
1 # 普通边：A 执行完必定执行 B
2 workflow.add_edge("A", "B")
3
4 # 条件边：根据条件决定下一步
5 workflow.add_conditional_edges(
6     "A",
7     决策函数,
8     {"条件1": "B", "条件2": "C"}
9 )
```

第四章 第一个程序

让我们写一个最简单的计数器程序，理解状态是怎么流转的。

简单线性流程



State 流转过程

初始: count=5 → count=6 → count=12 → 输出
add_edge(START, "increment") → add_edge("increment", "double") → add_edge("double", END)

完整代码

代码块

```
1  from langgraph.graph import StateGraph, START, END
2  from typing_extensions import TypedDict
3
4  # 1. 定义状态结构
5  class CounterState(TypedDict):
6      count: int
7      history: list[str]
8
9  # 2. 定义节点函数
10 def increment_node(state: CounterState):
11     """计数器 +1"""
12     new_count = state["count"] + 1
13     message = f"count: {state['count']} → {new_count}"
14
15     return {
16         "count": new_count,
17         "history": state["history"] + [message]
18     }
19
20 def double_node(state: CounterState):
21     """计数器翻倍"""
22     new_count = state["count"] * 2
23     message = f"count: {state['count']} → {new_count}"
24
25     return {
26         "count": new_count,
27         "history": state["history"] + [message]
28     }
29
30 def report_node(state: CounterState):
31     """报告结果"""
32     return {"history": state["history"] + [f"最终结果: {state['count']}"]}
33
```



```
34  # 3. 创建图
35  workflow = StateGraph(CounterState)
36
37  # 4. 添加节点
38  workflow.add_node("increment", increment_node)
39  workflow.add_node("double", double_node)
40  workflow.add_node("report", report_node)
41
42  # 5. 连接节点
43  workflow.add_edge(START, "increment")
44  workflow.add_edge("increment", "double")
45  workflow.add_edge("double", "report")
46  workflow.add_edge("report", END)
47
48  # 6. 编译
49  app = workflow.compile()
50
51  # 7. 运行
52  result = app.invoke({
53      "count": 5,
54      "history": ["开始执行"]
55  })
56
57  print("执行历史:")
58  for step in result["history"]:
59      print(f"    - {step}")
60  print(f"\n最终计数: {result['count']}")
```

运行结果

代码块

```
1  执行历史:
2    - 开始执行
3    - count: 5 → 6
4    - count: 6 → 12
5    - 最终结果: 12
6
7  最终计数: 12
```

关键点

1. **只返回需要更新的字段**: 节点不用返回完整的 state，只返回变化的部分
2. **状态自动传递**: 每个节点执行完，更新后的状态自动传给下一个节点
3. **必须编译**: 调用 `compile()` 后才能运行

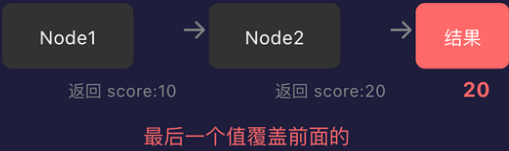
第五章 Reducer 机制

这是很多人一开始搞不明白的点：**为什么有时候状态会被覆盖，有时候会累加？**

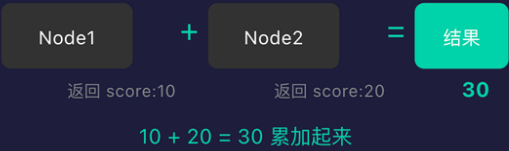
答案就是 **Reducer**。

Reducer 机制详解

没有 Reducer (覆盖模式)



使用 Reducer (累加模式)



代码对比

普通定义 (会被覆盖)

```
class State(TypedDict):  
    score: int  
    每次返回新值会覆盖旧值
```

使用Reducer (会累加)

```
class State(TypedDict):  
    score: Annotated[int, operator.add]  
    每次返回新值会和旧值相加
```

问题：状态被覆盖

默认情况下，每个节点返回的值会覆盖之前的值：

代码块

```
1 class State(TypedDict):  
2     score: int # 普通定义  
3  
4 # Node1 返回 {"score": 10} → state["score"] = 10  
5 # Node2 返回 {"score": 20} → state["score"] = 20 (覆盖了!)  
6 # 最终 score = 20
```

解决方案：使用 Reducer

代码块

```
1 from typing import Annotated  
2 import operator  
3  
4 class State(TypedDict):  
5     score: Annotated[int, operator.add] # 使用 operator.add 作为  
6     reducer  
7  
8 # Node1 返回 {"score": 10} → state["score"] = 0 + 10 = 10  
9 # Node2 返回 {"score": 20} → state["score"] = 10 + 20 = 30  
10 # 最终 score = 30 (累加!)
```

实际例子：游戏得分

代码块

```
1 from langgraph.graph import StateGraph, START, END  
2 from typing_extensions import TypedDict  
3 from typing import Annotated  
4 import operator
```

```
5
6 class ScoreState(TypedDict):
7     player_name: str # 普通字段，会被覆盖
8     score: Annotated[int, operator.add] # 累加
9     actions: Annotated[list[str], operator.add] # 列表也能累加
10
11 def level1_node(state):
12     return {"score": 10, "actions": ["通关第一关"]}
13
14 def level2_node(state):
15     return {"score": 20, "actions": ["通关第二关"]}
16
17 def level3_node(state):
18     return {"score": 30, "actions": ["通关第三关"]}
19
20 # 构建图...
21 workflow = StateGraph(ScoreState)
22 workflow.add_node("level1", level1_node)
23 workflow.add_node("level2", level2_node)
24 workflow.add_node("level3", level3_node)
25 workflow.add_edge(START, "level1")
26 workflow.add_edge("level1", "level2")
27 workflow.add_edge("level2", "level3")
28 workflow.add_edge("level3", END)
29
30 app = workflow.compile()
31 result = app.invoke({"player_name": "小明", "score": 0, "actions":
32 []})
33
34 print(f'玩家: {result['player_name']}')
35 print(f'总分: {result['score']}') # 10+20+30=60
36 print(f'完成动作: {result['actions']}')
```

常用 Reducer

Reducer	作用	适用场景
operator.add	数值相加	计数、得分
operator.add	列表拼接	历史记录、消息列表
add_messages	消息智能合并	聊天历史

第六章 条件分支

真实的应用不可能一条路走到黑，肯定需要根据情况走不同的路。



关键函数：add_conditional_edges

代码块

```
1 workflow.add_conditional_edges(  
2     "source_node",    # 从哪个节点出发  
3     routing_function, # 用什么函数决定路由  
4     {                 # 路由映射表  
5         "result1": "target_node1",  
6         "result2": "target_node2",  
7     }  
8 )
```

实际例子：智能客服路由

代码块

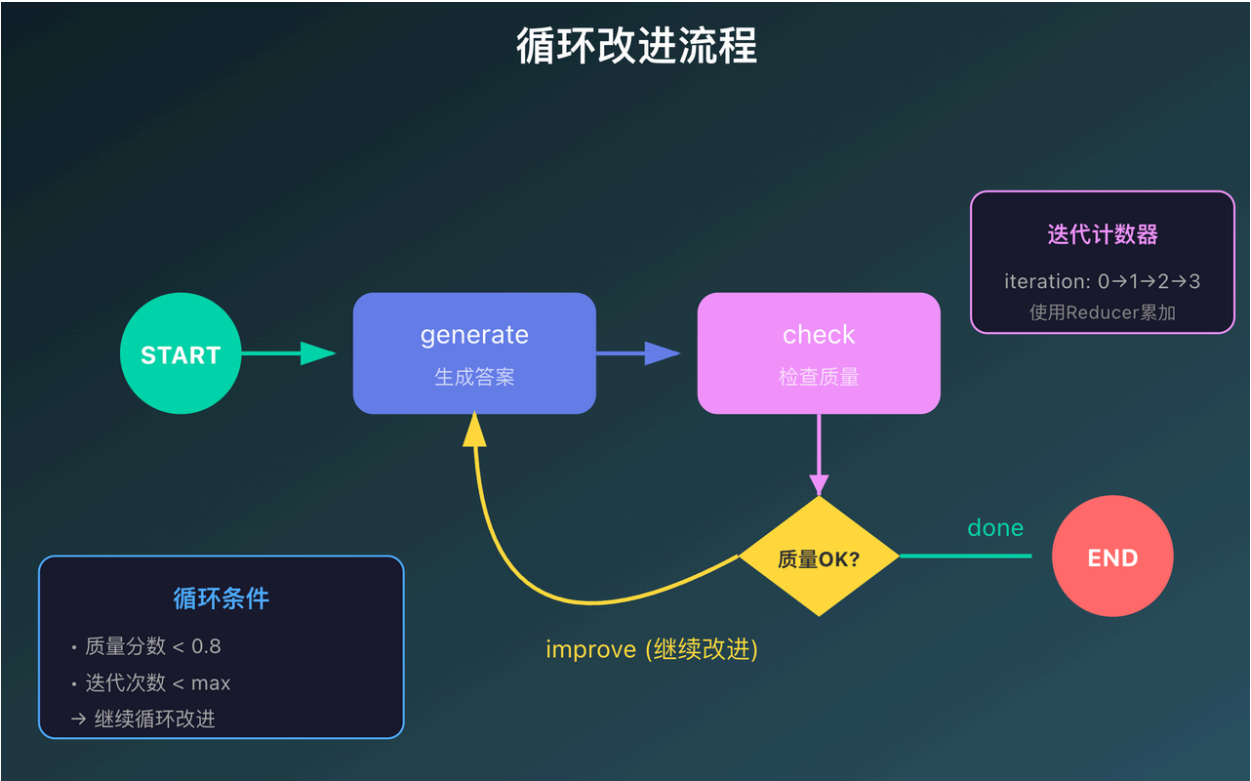
```
1 from langgraph.graph import StateGraph, START, END  
2 from typing_extensions import TypedDict  
3  
4 class CustomerState(TypedDict):  
5     question: str  
6     category: str  
7     answer: str  
8  
9 # 分类节点  
10 def classify_question(state: CustomerState):  
11     question = state["question"].lower()  
12  
13     if "退款" in question or "退货" in question:  
14         category = "refund"  
15     elif "物流" in question or "配送" in question:
```

```
16         category = "shipping"
17     elif "产品" in question or "使用" in question:
18         category = "product"
19     else:
20         category = "general"
21
22     return {"category": category}
23
24 # 处理节点
25 def handle_refund(state):
26     return {"answer": "退款专员：请提供订单号，3个工作日内处理。"}
27
28 def handle_shipping(state):
29     return {"answer": "物流客服：您的包裹正在配送中。"}
30
31 def handle_product(state):
32     return {"answer": "产品顾问：请查看说明书第3页。"}
33
34 def handle_general(state):
35     return {"answer": "客服：感谢咨询，还有其他问题吗？"}
36
37 # 路由函数
38 def route_question(state: CustomerState) -> str:
39     return state["category"]
40
41 # 构建图
42 workflow = StateGraph(CustomerState)
43
44 workflow.add_node("classify", classify_question)
45 workflow.add_node("refund", handle_refund)
46 workflow.add_node("shipping", handle_shipping)
47 workflow.add_node("product", handle_product)
48 workflow.add_node("general", handle_general)
49
50 workflow.add_edge(START, "classify")
51
52 # 关键：条件路由
53 workflow.add_conditional_edges(
54     "classify",
55     route_question,
56     {
57         "refund": "refund",
58         "shipping": "shipping",
59         "product": "product",
60         "general": "general"
61     }
62 )
63
64 # 所有分支都到 END
65 workflow.add_edge("refund", END)
66 workflow.add_edge("shipping", END)
67 workflow.add_edge("product", END)
68 workflow.add_edge("general", END)
69
70 app = workflow.compile()
71
72 # 测试
73 test_questions = [
74     "我想申请退款",
75     "我的快递到哪了？",
76     "这个产品怎么使用？",
77 ]
```

```
78
79 for q in test_questions:
80     result = app.invoke({"question": q, "category": "", "answer":
      ""})
81     print(f"问题: {q}")
82     print(f"回答: {result['answer']}\n")
```

第七章 循环流程

有时候一次处理不够好，需要反复迭代优化。LangGraph 天然支持循环。



关键点

- 1. 条件边可以指回前面的节点：形成循环
- 2. 必须有退出条件：否则会无限循环
- 3. 建议设置最大迭代次数：兜底保护

实际例子：答案优化器

代码块

```
1 from langgraph.graph import StateGraph, START, END
2 from typing_extensions import TypedDict
3 from typing import Annotated
4 import operator
5
6 class ImprovementState(TypedDict):
7     question: str
8     answer: str
9     quality_score: float
10    iteration: Annotated[int, operator.add]
```

| return state



刘家成 3月9日 22:57
不应该返回 state，而是应该返回一个 None 或者空字典



户一帆 4月12日 22:47
先执行了 generate_answer，state 非空

```

11     max_iterations: int
12
13 def generate_answer(state: ImprovementState):
14     """生成或改进答案"""
15     iteration = state.get("iteration", 0)
16
17     # 模拟答案逐步改进
18     if iteration == 0:
19         answer = "简单回答：这是一个基础答案。"
20         score = 0.5
21     elif iteration == 1:
22         answer = "改进回答：这是一个更详细的答案，包含更多信息。"
23         score = 0.7
24     else:
25         answer = "完善回答：经过仔细思考的完整答案，包含背景和建议。"
26         score = 0.9
27
28     print(f"第{iteration + 1}次生成 (质量: {score})")
29
30     return {
31         "answer": answer,
32         "quality_score": score,
33         "iteration": 1 # 每次+1 (因为有 reducer)
34     }
35
36 def check_quality(state):
37     """检查质量，不更新状态"""
38     return state
39
40 def should_continue(state: ImprovementState) -> str:
41     """决定是继续还是结束"""
42     if state["quality_score"] >= 0.8:
43         return "done"
44     elif state["iteration"] >= state["max_iterations"]:
45         return "done"
46     else:
47         return "improve"
48
49 # 构建带循环的图
50 workflow = StateGraph(ImprovementState)
51
52 workflow.add_node("generate", generate_answer)
53 workflow.add_node("check", check_quality)
54
55 workflow.add_edge(START, "generate")
56 workflow.add_edge("generate", "check")
57
58 # 关键：条件边可以循环回去
59 workflow.add_conditional_edges(
60     "check",
61     should_continue,
62     {
63         "improve": "generate", # 循环
64         "done": END # 结束
65     }
66 )
67
68 app = workflow.compile()
69
70 result = app.invoke({
71     "question": "什么是人工智能？ ",
72     "answer": "",

```

| ChatState(MessagesState):



李楠 2月10日 19:39

- 继承 MessagesState：自带 messages: Annotated[list, add_messages]，即消息列表，且带“追加”语义（新消息会 append，不会覆盖）。

```
73     "quality_score": 0.0,
74     "iteration": 0,
75     "max_iterations": 5
76 })
77
78 print(f"\n最终答案: {result['answer']}")
79 print(f"最终质量: {result['quality_score']}")
80 print(f"迭代次数: {result['iteration']}")
```

第八章 聊天机器人

做聊天机器人最头疼的是**消息历史管理**。LangGraph 提供了专门的工具来解决这个问题。

核心：MessagesState 和 add_messages

代码块

```
1  from langgraph.graph import MessagesState
2  from langgraph.graph.message import add_messages
3  from langchain_core.messages import HumanMessage, AIMessage
4
5  # 方式1: 继承 MessagesState
6  class ChatState(MessagesState):
7      user_name: str
8      conversation_count: int
9
10 # 方式2: 手动定义
11 class ChatState(TypedDict):
12     messages: Annotated[list[BaseMessage], add_messages]
13     user_name: str
```

`add_messages` 是一个智能 Reducer，它会：

- 自动追加新消息
- 处理消息 ID 去重
- 保持消息顺序

完整聊天机器人

代码块

```
1  import os
2  from langgraph.graph import StateGraph, START, END, MessagesState
3  from langchain_core.messages import HumanMessage, AIMessage,
4  SystemMessage
5  from langchain_community.llms import Tongyi
6
7  os.environ["DASHSCOPE_API_KEY"] = "your-key"
8
9  llm = Tongyi(model="qwen-plus", temperature=0.7)
10
11 class ChatState(MessagesState):
12     user_name: str
13     conversation_count: int
14
15 def chatbot_node(state: ChatState):
```

```
15     """聊天节点"""
16     # 构建系统提示
17     system_prompt = f"""你是一个友好的AI助手。
18 用户名: {state.get('user_name', '用户')}
19 当前是第 {state.get('conversation_count', 0)} 轮对话。"""
20
21     messages = [SystemMessage(content=system_prompt)] +
state["messages"]
22     response = llm.invoke(messages)
23
24     return {
25         "messages": [AIMessage(content=response)],
26         "conversation_count": state.get("conversation_count", 0) +
1
27     }
28
29     # 构建图
30     workflow = StateGraph(ChatState)
31     workflow.add_node("chatbot", chatbot_node)
32     workflow.add_edge(START, "chatbot")
33     workflow.add_edge("chatbot", END)
34     app = workflow.compile()
35
36     # 多轮对话
37     state = {"messages": [], "user_name": "小明", "conversation_count":
0}
38
39     conversations = [
40         "你好，我是小明",
41         "我刚才说我叫什么？ ",
42         "帮我推荐一本Python书"
43     ]
44
45     for user_input in conversations:
46         print(f"用户: {user_input}")
47         state["messages"].append(HumanMessage(content=user_input))
48         result = app.invoke(state)
49         state = result
50         print(f"助手: {result['messages'][-1].content}\n")
```

第九章 Agent 工具调用

Agent 和普通聊天机器人的区别是：**Agent 能使用工具**。

定义工具

使用 `@tool` 装饰器：

代码块

```
1  from langchain_core.tools import tool
2
3  @tool
4  def search_web(query: str) -> str:
5      """搜索网络信息
6
7      Args:
8          query: 搜索关键词
9      """
```



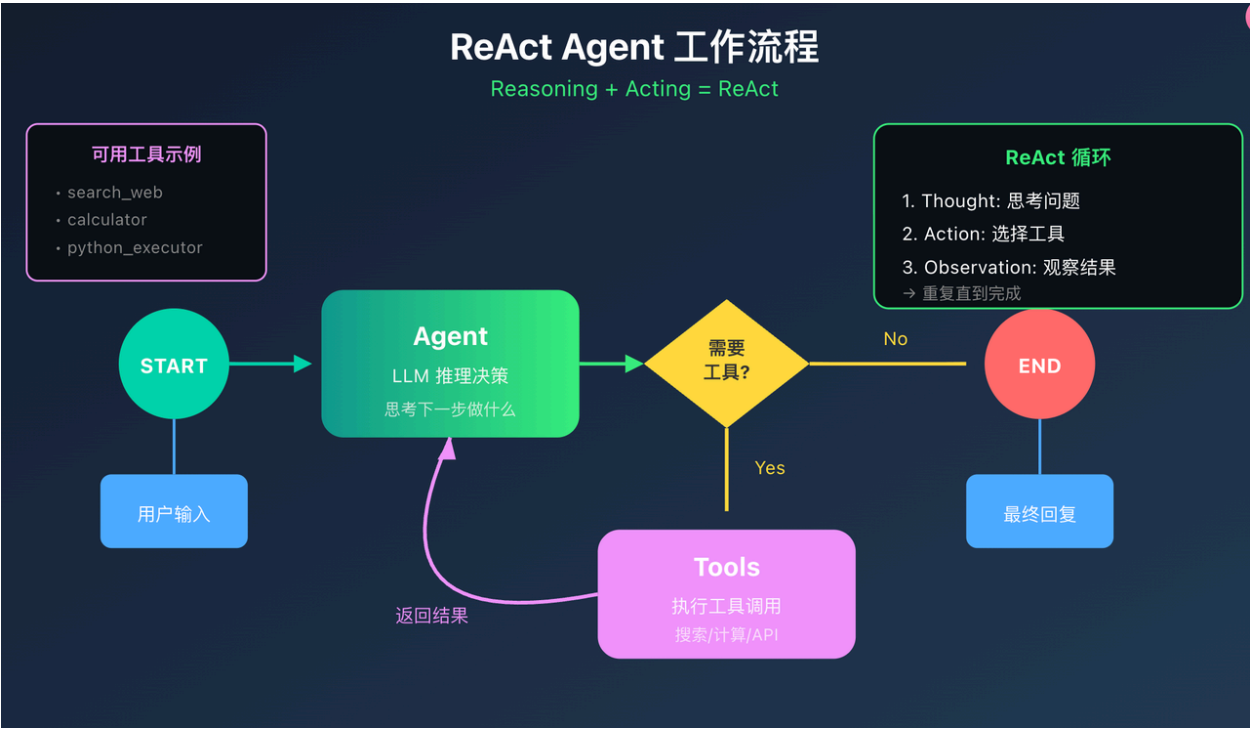
```
10     # 实际实现
11     return f"搜索 '{query}' 的结果: ..."
12
13 @tool
14 def calculator(expression: str) -> str:
15     """计算数学表达式
16
17     Args:
18         expression: 数学表达式, 如 "2+2"
19     """
20     try:
21         result = eval(expression)
22         return f"计算结果: {expression} = {result}"
23     except:
24         return "计算错误"
25
26 @tool
27 def get_weather(city: str) -> str:
28     """查询城市天气
29
30     Args:
31         city: 城市名称
32     """
33     weather_data = {
34         "北京": "晴天, 15-25度",
35         "上海": "多云, 18-28度",
36     }
37     return weather_data.get(city, f"{city}天气信息不可用")
```

工具定义要点

- 1. 必须有 **docstring**: 描述工具用途, AI 靠这个判断什么时候用
- 2. 参数要有类型注解: 让 AI 知道怎么传参
- 3. **Args** 部分详细说明: 每个参数是干什么的

第十章 ReAct Agent

ReAct = Reasoning + Acting, 是目前最流行的 Agent 架构。



工作流程

1. **Thought**: AI 思考需要做什么
2. **Action**: 选择并调用工具
3. **Observation**: 观察工具返回结果
4. **重复**: 直到任务完成

使用预置组件

LangGraph 提供了现成的组件：

代码块

```
1  from langgraph.prebuilt import create_react_agent, ToolNode
2  from langchain_community.chat_models import ChatTongyi
3  from langchain_core.tools import tool
4  from langchain_core.messages import SystemMessage
5
6  @tool
7  def add(a: float, b: float) -> float:
8      """两数相加"""
9      return a + b
10
11  @tool
12  def multiply(a: float, b: float) -> float:
13      """两数相乘"""
14      return a * b
15
16  # 创建模型
17  llm = ChatTongyi(model="qwen-plus", temperature=0.7)
18
19  # 创建 ReAct Agent
20  agent = create_react_agent(
21      model=llm,
22      tools=[add, multiply],
23      messages_modifier=SystemMessage(content="你是一个数学计算助手。")
```

```
24 )
25
26 # 使用
27 result = agent.invoke({
28     "messages": [{"role": "user", "content": "计算 (10 + 5) × 3"}]
29 })
30 print(result["messages"][-1].content)
```

手动实现 ReAct Agent

如果需要完全控制，可以手动实现：

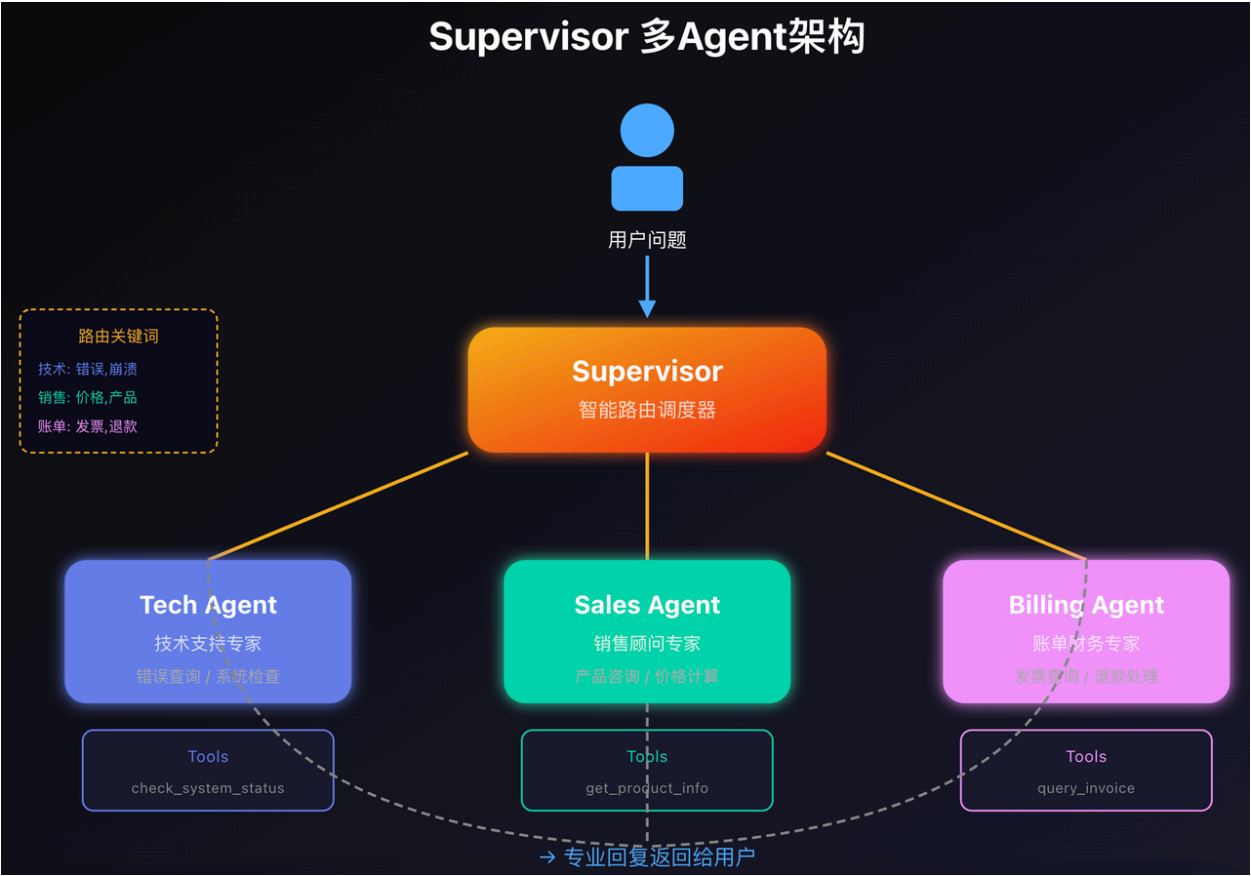
代码块

```
1  from langgraph.graph import StateGraph, START, END
2  from langgraph.graph.message import add_messages
3  from langgraph.prebuilt import ToolNode
4  from typing import Annotated, Literal
5  from typing_extensions import TypedDict
6
7  class AgentState(TypedDict):
8      messages: Annotated[list, add_messages]
9      iterations: Annotated[int, operator.add]
10
11  # 工具定义...
12  tools = [python_executor, text_analyzer]
13  tool_node = ToolNode(tools)
14
15  # 模型绑定工具
16  llm_with_tools = llm.bind_tools(tools)
17
18  def call_model(state: AgentState):
19      """调用模型决策"""
20      messages = state["messages"]
21      response = llm_with_tools.invoke(messages)
22      return {"messages": [response], "iterations": 1}
23
24  def should_continue(state) -> Literal["tools", "__end__"]:
25      """判断是否需要调用工具"""
26      last_message = state["messages"][-1]
27      if hasattr(last_message, "tool_calls") and
last_message.tool_calls:
28          return "tools"
29          return "__end__"
30
31  # 构建图
32  workflow = StateGraph(AgentState)
33  workflow.add_node("agent", call_model)
34  workflow.add_node("tools", tool_node)
35
36  workflow.add_edge(START, "agent")
37  workflow.add_conditional_edges("agent", should_continue, {
38      "tools": "tools",
39      "__end__": END
40  })
41  workflow.add_edge("tools", "agent")
42
43  app = workflow.compile()
```

第十一章 多Agent协作

当任务复杂时，一个 Agent 搞不定，需要多个专业 Agent 配合。

Supervisor 模式



核心思想：一个 Supervisor 负责调度，多个专业 Agent 负责执行。

完整实现

```
代码块
1  from langgraph.graph import StateGraph, START, END
2  from langchain_core.messages import HumanMessage, AIMessage,
   BaseMessage
3  from langchain_core.tools import tool
4  from typing import Annotated, Literal, Sequence
5  from typing_extensions import TypedDict
6  from langgraph.graph.message import add_messages
7
8  # 1. 定义状态
9  class SupervisorState(TypedDict):
10     messages: Annotated[Sequence[BaseMessage], add_messages]
11     next: str # 下一个执行的 Agent
12
13  # 2. 定义各种工具
14  @tool
15  def check_system_status(component: str) -> str:
16     """检查系统组件状态"""
17     status = {"database": "正常", "server": "负载高", "api": "正常"}
18     return status.get(component, "未知")
19
```

```
20 @tool
21 def get_product_info(product_name: str) -> str:
22     """获取产品信息"""
23     products = {
24         "基础版": "99元/月, 基础功能",
25         "专业版": "299元/月, 高级功能",
26     }
27     return products.get(product_name, "产品不存在")
28
29 @tool
30 def query_invoice(order_id: str) -> str:
31     """查询发票"""
32     invoices = {"ORD001": "已开具", "ORD002": "处理中"}
33     return invoices.get(order_id, "订单不存在")
34
35 # 3. 定义专业 Agent 节点
36 def tech_agent_node(state):
37     """技术支持 Agent"""
38     last_message = state["messages"][-1].content
39
40     if "系统" in last_message or "服务器" in last_message:
41         result = check_system_status.invoke({"component":
42 "server"})
43         response = f"【技术支持】系统状态: {result}"
44     else:
45         response = "【技术支持】请描述您遇到的技术问题。"
46
47     return {"messages": [AIMessage(content=response)]}
48
49 def sales_agent_node(state):
50     """销售 Agent"""
51     last_message = state["messages"][-1].content
52
53     for product in ["基础版", "专业版"]:
54         if product in last_message:
55             info = get_product_info.invoke({"product_name":
56 product})
57             return {"messages": [AIMessage(content=f"【销售顾问】
58 {info}")] }
59
60     return {"messages": [AIMessage(content="【销售顾问】 我们有基础版和
61 专业版, 需要了解哪个? ")] }
62
63 def billing_agent_node(state):
64     """账单 Agent"""
65     return {"messages": [AIMessage(content="【账单专员】 请提供订单号,
66 我来查询。")] }
67
68 # 4. Supervisor 节点
69 def supervisor_node(state):
70     """Supervisor: 决定下一步"""
71     last_message = state["messages"][-1].content
72
73     tech_keywords = ["错误", "bug", "崩溃", "系统", "服务器"]
74     sales_keywords = ["价格", "购买", "产品", "套餐"]
75     billing_keywords = ["发票", "支付", "退款", "账单"]
76
77     if any(kw in last_message for kw in tech_keywords):
78         next_agent = "tech_support"
79     elif any(kw in last_message for kw in sales_keywords):
80         next_agent = "sales"
81     elif any(kw in last_message for kw in billing_keywords):
```

```

77         next_agent = "billing"
78     else:
79         next_agent = "sales" # 默认销售
80
81     return {"next": next_agent}
82
83 # 5. 路由函数
84 def route_after_supervisor(state) -> Literal["tech_support",
85 "sales", "billing"]:
86     return state["next"]
87
88 # 6. 构建图
89 workflow = StateGraph(SupervisorState)
90
91 workflow.add_node("supervisor", supervisor_node)
92 workflow.add_node("tech_support", tech_agent_node)
93 workflow.add_node("sales", sales_agent_node)
94 workflow.add_node("billing", billing_agent_node)
95
96 workflow.add_edge(START, "supervisor")
97 workflow.add_conditional_edges(
98     "supervisor",
99     route_after_supervisor,
100     {
101         "tech_support": "tech_support",
102         "sales": "sales",
103         "billing": "billing",
104     }
105 )
106 workflow.add_edge("tech_support", END)
107 workflow.add_edge("sales", END)
108 workflow.add_edge("billing", END)
109
110 app = workflow.compile()
111
112 # 7. 测试
113 test_cases = [
114     "我遇到500错误，系统无法访问",
115     "专业版多少钱？",
116     "我需要查发票",
117 ]
118
119 for query in test_cases:
120     print(f"用户: {query}")
121     result = app.invoke({"messages":
122 [HumanMessage(content=query)], "next": ""})
123     for msg in result["messages"]:
124         if isinstance(msg, AIMessage):
125             print(msg.content)
126     print()

```

第十二章 实战项目

智能客服系统架构

把前面学的所有东西整合起来，做一个完整的智能客服系统：

1. Supervisor 分析问题类型

- 2. 技术支持 Agent 处理技术问题
- 3. 销售顾问 Agent 处理产品咨询
- 4. 账单专员 Agent 处理财务问题
- 5. 每个 Agent 都有自己的专业工具
- 6. 支持多轮对话和历史记忆

完整代码见附带的 `langgraph_supervisor_1.py` 文件。

第十三章 进阶话题

4. 持久化和记忆

使用 `checkpointer` 保存对话状态：

代码块

```
1  from langgraph.checkpoint.memory import MemorySaver
2
3  memory = MemorySaver()
4  app = workflow.compile(checkpointer=memory)
5
6  # 使用 thread_id 管理不同会话
7  config = {"configurable": {"thread_id": "user_123"}}
8  result = app.invoke(state, config=config)
```

5. 流式输出

实现打字机效果：

代码块

```
1  # 方式1: 使用 streaming=True 的模型
2  llm = Tongyi(model="qwen-plus", streaming=True)
3
4  for chunk in llm.stream(messages):
5      print(chunk, end="", flush=True)
6
7  # 方式2: 使用 app.stream()
8  for event in app.stream(initial_state):
9      print(event)
```

6. 子图 (Subgraph)

复杂系统可以拆成多个子图：

代码块

```
1  # 定义子图
2  sub_workflow = StateGraph(SubState)
3  # ... 添加节点和边
4  sub_graph = sub_workflow.compile()
5
6  # 在主图中使用子图作为节点
7  main_workflow.add_node("sub_process", sub_graph)
```

7. Human-in-the-Loop

让人类参与决策：

代码块

```
1  # 设置断点
2  app = workflow.compile(
3      checkpointer=memory,
4      interrupt_before=["sensitive_action"]
5  )
6
7  # 运行到断点会暂停
8  result = app.invoke(state, config)
9
10 # 人工审核后继续
11 app.invoke(None, config) # 继续执行
```

8. 调试技巧

代码块

```
1  # 可视化图结构
2  print(app.get_graph().draw_mermaid())
3
4  # 打印执行过程
5  for event in app.stream(state, stream_mode="debug"):
6      print(event)
```